

미션 가이드 (모바일 물리 퍼즐 + 자동 검증 솔버)

■ 미션 개요	
발주사	(주)링크즈
프로젝트명	물리 퍼즐 + 자동 검증 솔버 (Physics Puzzle with Auto-Solver)
장르	2D 물리 퍼즐 / 싱글 플레이
플랫폼	Android 모바일 (스마트폰 기준, 세로 모드 고정 · 한 손 조작)
출시 전제	완성된 결과물은 학생 분들의 동의 하에 (주)링크즈 명의로 Google Play 스토어 출시를 검토합니다. 이후 이 출시 링크로 포트폴리오로 이용하실 수 있도록 도와드리겠습니다.
사용 엔진	Unity 3D, C# / URP (Mobile) / IL2CPP + ARM64

■ 프로그램 소개
손으로 그린 것이 그대로 물리 법칙을 따르는 즐거움이 핵심 방향성입니다. 플레이어는 화면에 선을 그어 물체를 만들고, 공을 목표 지점까지 굴려 보냅니다. 정답이 하나가 아니라는 것이 이 장르의 재미이며, 어설프게 그린 다리가 우연히 성공하는 순간이 가장 즐거운 순간입니다. 그래서 실패를 가볍게 만들고 재시도를 빠르게 만드는 것이 기획의 출발점입니다. 이 미션의 진짜 난이도는 레벨을 만드는 일에 있습니다. 물리 퍼즐은 만든 사람도 풀 수 있는지 확인하기 어렵습니다. 레벨 100개를 손으로 하나씩 검수하면 두 달이 다 갑니다. 그래서 이 프로젝트는 게임과 함께 도구를 만듭니다. 만든 레벨을 프로그램이 스스로 수없이 시도해 보고, 풀리는지 아닌지와 얼마나 어려운지를 판정하는 자동 검증 솔버가 요구 범위에 포함됩니다. 솔버가 성립하려면 물리 시뮬레이션이 매번 같은 결과를 내야 합니다. 같은 입력을 넣었는데 결과가 달라지면 검증 자체가 의미를 잃습니다. 결정론을 확보하는 것이 이 프로젝트에서 가장 먼저 해결해야 할 문제입니다. 모바일은 플레이 맥락도 다릅니다. 한 판이 30초에서 2분 안에 끝나야 하고, 실패한 뒤 다시 시작하기까지 한 번의 탭이면 충분해야 합니다. 한 손으로 지하철에서 할 수 있어야 한다는 것이 이 프로젝트의 조작 설계 기준입니다. 콘텐츠 확장은 코드가 아니라 데이터로 이루어져야 합니다. 신규 레벨과 장치는 데이터 추가만으로 동작해야 하며, 최종 발표 현장에서 즉석으로 레벨을 하나 만들어 솔버가 판정하는 과정을 확인할 예정입니다.

■ 레퍼런스
- Crayon Physics Deluxe — 그린 것이 물체가 되는 조작의 원형 - Cut the Rope — 물리 퍼즐의 터치 조작과 별 3 개 목표 구조. 모바일 UI 기준으로 참고할 것 - Angry Birds — 실패를 가볍게 만들고 재시도를 빠르게 만드는 흐름 - Poly Bridge — 만든 구조물을 시뮬레이션으로 검증하는 흐름과 결과 표현

■ 퍼즐 요소 구성	
아래 구성은 명칭과 콘셉트, 기능은 기획에 맞게 변경할 수 있으며 예시로 참고해주세요.	
플레이어 도구 3종	① 고정 선 — 그리면 그 자리에 붙박이로 남는 발판 ② 자유 물체 — 그리면 중력의 영향을 받아 떨어지는 물체 ③ 회전축 — 두 물체를 이어 지렛대나 바퀴를 만들
레벨 장치 6종	① 목표 지점 — 공이 도달하면 클리어 ② 별 — 선택 수집 목표, 3개 배치 ③ 장애물 — 닿으면 실패하는 가시나 낭떠러지 ④ 스위치와 문 — 조건을 만족해야 길이 열림 ⑤ 바람 구역 — 지정 방향으로 힘을 가함 ⑥ 폭탄 — 일정 시간 뒤 주변을 밀어냄
잉크 제한	한 레벨에서 그릴 수 있는 총 길이를 제한합니다. 제한이 없으면 화면을 가득 채우는 것이 최적해가 되어 퍼즐이 성립하지 않습니다.
레벨 수	40개 내외. 이 중 최소 10개는 레벨 에디터로 만들고 솔버로 검증한 것이어야 합니다.
■ 모바일 조작 설계 (필수)	
이 미션에서 기획팀이 가장 많은 시간을 써야 하는 영역입니다. 아래는 최소 요구사항이며, 더 나은 방식을 찾았다면 근거와 함께 대체할 수 있습니다.	
그리기	손가락으로 꺾적을 그으면 그대로 물체가 됩니다. 그리는 동안 손가락에 가려지는 부분이 생기므로, 꺾적을 굵게 표시하거나 그림자를 함께 그려 위치를 알 수 있게 해야 합니다.
잉크 표시	남은 잉크가 그리는 도중에도 실시간으로 보여야 합니다. 다 쓰고 나서야 알게 되는 구조는 좌절을 만듭니다.
되돌리기와 초기화	직전에 그린 것 하나를 지우는 되돌리기와, 레벨 전체를 처음으로 되돌리는 초기화를 각각 한 번의 탭으로 제공합니다.
시뮬레이션 제어	그리기를 마친 뒤 시작 버튼으로 물리 시뮬레이션을 시작합니다. 시작 전에는 시간이 멈춰 있어야 하며, 진행 중 일시정지와 배속을 지원합니다.
재시도	실패 후 한 번의 탭으로 다시 시작할 수 있어야 합니다. 결과 화면을 거쳐야만 재시도할 수 있는 구조는 사용하지 않습니다.
UI 규격	터치 타겟은 최소 48dp를 확보하고, 노치·펀치홀·제스처 바를 피하도록 Safe Area를 적용합니다. 주요 조작 UI는 엄지가 닿는 화면 하단에 배치합니다.
검증 방법	게임을 처음 접한 사람에게 기기를 건네고, 튜토리얼만 본 상태에서 초반 레벨 세 개를 스스로 클리어할 수 있는지 확인합니다.

■ 필수 유저 체험 및 기획 요소	
정답이 하나가 아닌 문제를 내고, 실패를 가볍게 만들어 계속 시도하게 만드는 경험을 구현합니다.	
발상 재미	한 레벨을 푸는 방법이 여러 가지여야 합니다. 의도한 정답 외의 풀이가 나오는 것은 버그가 아니라 성과이며, 솔버가 이를 찾아내 보여줄 수 있어야 합니다.
즉시 재시도 재미	실패해도 손해가 없어야 합니다. 한 번의 탭으로 처음 상태로 돌아가고, 곧바로 다시 그릴 수 있어야 합니다.
관찰 재미	그린 물체가 실제로 휘고 넘어지고 굴러가는 것이 보여야 합니다. 결과만 판정하고 과정을 생략하는 표현은 이 장르의 핵심 재미를 훼손합니다.
속련 재미	잉크를 적게 쓰거나 더 빨리 도달하면 더 좋은 평가를 받아야 합니다. 클리어만이 목표가 되면 다시 할 이유가 사라집니다.
짧은 세션 재미	한 레벨이 30초에서 2분 안에 끝나야 합니다. 모바일 플레이어는 한 세션을 길게 잡지 않습니다.
성장 재미	레벨을 진행하며 새로운 도구와 장치가 순차적으로 해금됩니다. 최소 8종 이상의 해금 항목이 있어야 합니다.
창작 재미	레벨 에디터를 플레이어에게 공개할지 여부는 기획 판단에 맡깁니다. 다만 개발용 에디터는 반드시 있어야 합니다.
■ 필요로 하는 기능	
물리 시뮬레이션의 재현성이 전제이며, 그 위에서 레벨을 자동으로 검증하는 도구를 함께 만드는 것이 이 미션의 핵심입니다.	
기능 항목	참고
터치 드로잉	손가락 꺾적을 물체로 변환해야 함. 점이 지나치게 촘촘하지 않도록 꺾적을 단순화하고, 자기 교차나 지나치게 짧은 선에 대한 처리를 포함할 것.
결정론적 물리 시뮬레이션	같은 입력에서 항상 같은 결과가 나와야 함. 고정 시간 간격으로 물리를 갱신하며, 프레임 속도가 결과에 영향을 주면 안 됨.
잉크 제한과 되돌리기	그린 길이의 합을 관리하고, 되돌리기와 초기화가 물리 상태까지 정확히 복원해야 함.
레벨 데이터	레벨 구성이 외부 데이터로 정의되고, 데이터 추가만으로 동작해야 함. 레벨 추가에 코드 수정이 필요하다면 요구사항 미충족으로 간주함.
레벨 에디터	장치를 배치하고 목표와 잉크 제한을 설정해 레벨 데이터를 만들 수 있는 개발용 도구.

자동 검증 솔버	만든 레벨에 대해 프로그램이 스스로 여러 가지 풀이를 시도하고, 클리어 가능 여부와 성공률을 산출해야 함. 이 미션의 핵심 요구사항임.
난이도 자동 분류	솔버가 산출한 성공률을 기준으로 레벨의 난이도를 분류하고, 풀리지 않는 레벨을 걸러낼 수 있어야 함.
리플레이	플레이어의 조작을 기록해 그대로 재생할 수 있어야 함. 결정론이 지켜지는지 확인하는 수단이자 발표 자료로 사용함.
자동 저장 및 복구	앱 전환이나 강제 종료 시에도 진행도가 손실 없이 복원되어야 함.
튜토리얼	첫 레벨에서 그리기 → 시작 → 실패 → 되돌리기 → 재시도까지 직접 해보게 하는 인터랙티브 튜토리얼이 필요함.

■ 활용해야 하는 기술

- 고정 시간 간격 물리 갱신과 결정론 확보 — 프레임 속도와 시뮬레이션 결과를 분리할 것 - 손가락 궤적 단순화 알고리즘과 다각형 분할을 통한 충돌체 생성 - 화면 표시 없이 물리만 빠르게 반복 실행하는 검증 모드 구성 - 시드 고정 난수와 입력 기록·재생을 이용한 리플레이 구현 - JSON 직렬화를 이용한 레벨 데이터 및 진행도 저장 - 오브젝트 풀링과 물리 연산 부하 관리, Application.targetFrameRate 고정 - UI 화면 Safe Area 대응 및 세로 모드 레이아웃 구성 - WBS — 일정 관리 및 리스크 관리 / 형상관리 — Git / Unity 3D, C#	
--	--

■ 개발환경

실무와 동일한 환경을 체험하기 위해서 회사에서 적용중인 개발환경입니다.	
---	--

빌드환경	Unity 6000 버전 LTS
이슈관리	디스코드, Jira, Notion, Slack, 텔레그램, 카카오톡 등 자유 선택 가능
문서관리	Notion, Microsoft Office, Google Docs
형상관리	GitHub. 브랜치 전략 수립과 코드 리뷰 이력이 남아야 함 (svn 사용도 무방)

■ 개발 환경 요건

빌드 형식	APK (IL2CPP + ARM64, 64비트 지원 필수)
Target API	현재 신규 앱 및 업데이트는 Android 15 (API 35) 이상을 타겟해야 합니다.
최소 지원	Android 8.0 (API 26) 이상 권장.

스토어 자산	앱 아이콘 512×512 / 그래픽 이미지 1024×500 스마트폰 스크린샷 최소 2장 (권장 4~8장), 태블릿 스크린샷 권장 짧은 설명 80자 이내, 자세한 설명 4,000자 이내, 프로모션 영상(선택)
■ 결과물	
• 게임 기획서 (코어 루프, 도구·장치 규칙, 터치 조작 설계, 화면 설계) • 솔버 설계 문서 (풀이 시도 방식, 성공률 산출 기준, 난이도 분류 기준) • 레벨 데이터 및 솔버 검증 결과표 (레벨별 성공률과 난이도, 엑셀 문서) • 레벨 에디터 (개발용 도구, 실행 가능한 형태) • 실기기 빌드 (APK) • 아이콘, 그래픽 이미지, 스크린샷, 설명 문안 • 프로젝트 일정 관리 문서 혹은 기록 문서 • 사운드, 그래픽 등의 리소스 • 시연 영상 (2 분 내외, 실기기 화면 녹화)	